

Tvorba internetových aplikací

June 6, 2026

1 Základy internetových aplikací

1.1 Charakteristika internetovej aplikácie

Internetová aplikácia je aplikácia, ku ktorej používateľ pristupuje prostredníctvom webového prehliadača. Typicky je založená na architektúre klient–server. Klientom je webový prehliadač, ktorý zobrazuje používateľské rozhranie a vykonáva klientsky kód, najmä JavaScript. Server spracúva požiadavky, pristupuje k databáze, uchováva stav aplikácie a generuje odpovede.

Základné technológie webovej aplikácie sú:

- **HTTP** – protokol na komunikáciu medzi klientom a serverom,
- **HTML** – jazyk na štruktúrovanie obsahu stránky,
- **CSS** – jazyk na opis vzhľadu a rozloženia stránky,
- **SVG** – XML formát pre vektorovú grafiku,
- **JavaScript** – programovací jazyk na strane klienta.

1.2 HTTP

HTTP, teda HyperText Transfer Protocol, je aplikačný protokol používaný na prenos webových dokumentov. Funguje na princípe požiadavka–odpoveď. Klient pošle serveru HTTP požiadavku a server vráti HTTP odpoveď.

HTTP požiadavka obsahuje najmä:

- metódu, napríklad GET, POST, PUT, DELETE,
- URL požadovaného zdroja,
- hlavičky,
- voliteľné telo požiadavky.

Najčastejšie metódy sú:

- **GET** – získanie zdroja zo servera,

- POST – odoslanie dát na server, napríklad formulára,
- PUT – nahradenie alebo vytvorenie zdroja,
- DELETE – vymazanie zdroja.

HTTP odpoveď obsahuje stavový kód, hlavičky a telo odpovede. Stavové kódy sa delia napríklad na:

- 2xx – úspech, napríklad 200 OK,
- 3xx – presmerovanie,
- 4xx – chyba klienta, napríklad 404 Not Found,
- 5xx – chyba servera.

HTTP je bezstavový protokol. To znamená, že samotný protokol si medzi požiadavkami nepamätá používateľa ani stav aplikácie. Stav sa preto rieši pomocou cookies, sessions, tokenov alebo iných mechanizmov.

1.3 HTML

HTML, teda HyperText Markup Language, opisuje štruktúru webovej stránky. Pomocou značiek definuje nadpisy, odseky, odkazy, formuláre, tabuľky, obrázky a ďalšie prvky dokumentu.

Príklad jednoduchkej HTML stránky:

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>Ukážka</title>
</head>
<body>
  <h1>Nadpis stránky</h1>
  <p>Toto je odsek textu.</p>
</body>
</html>
```

HTML by malo niesť najmä význam a štruktúru obsahu, nie jeho vizuálnu prezentáciu. Na vzhľad sa používa CSS a na správanie JavaScript.

1.4 CSS

CSS, teda Cascading Style Sheets, slúži na opis vzhľadu HTML dokumentu. Určuje farby, písma, rozmery, rozmiestnenie prvkov, animácie a responzívne správanie stránky.

Príklad:

```
body {
    font-family: sans-serif;
    margin: 2em;
}

h1 {
    color: navy;
}
```

Výhodou CSS je oddelenie obsahu od prezentácie. Ten istý HTML dokument môže mať rôzne vizuálne štýly bez toho, aby sa menila jeho štruktúra.

1.5 SVG

SVG, teda Scalable Vector Graphics, je XML formát pre vektorovú grafiku. Na rozdiel od rastrových obrázkov SVG neopisuje pixely, ale geometrické tvary, napríklad čiary, kruhy, obdĺžniky, krivky a text.

Príklad SVG obrázka:

```
<svg xmlns="http://www.w3.org/2000/svg" width="200" height="100">
  <circle cx="50" cy="50" r="40"
    stroke="black" stroke-width="2" fill="red" />
</svg>
```

Výhody SVG:

- je nezávislé od rozlíšenia,
- je vhodné pre ikony, diagramy a grafy,
- SVG elementy sú súčasťou DOM,
- možno ich upravovať JavaScriptom,
- možno na ne pripájať udalosti,
- podporuje hyperlinky a animácie.

Nevýhodou SVG je, že pri veľmi zložitých obrázkoch s tisíckami uzlov môže byť pomalšie. Vtedy je vhodnejšie použiť napríklad `canvas` alebo bitmapové obrázky.

1.6 JavaScript

JavaScript je programovací jazyk používaný najmä na strane klienta. Umožňuje meniť HTML dokument, reagovať na udalosti používateľa, komunikovať so serverom a vytvárať interaktívne aplikácie.

JavaScript sa používa napríklad na:

- validáciu formulárov,

- manipuláciu s DOM,
- obsluhu udalostí,
- asynchrónnu komunikáciu so serverom,
- prácu s JSON dátami,
- animácie a interaktívne komponenty.

Príklad manipulácie s DOM:

```
document.getElementById("message").innerHTML = "Ahoj svet!";
```

Moderný JavaScript obsahuje konštrukcie ako `let`, `const`, arrow funkcie, triednu syntax, moduly a prácu s asynchrónnym kódom.

1.7 Oddelenie prezentácie, obsahu a kódu

Pri tvorbe internetových aplikácií je dôležité oddeliť:

- **obsah** – HTML,
- **prezentáciu** – CSS,
- **správanie a logiku** – JavaScript.

Takéto oddelenie zlepšuje čitateľnosť, udržiavateľnosť a znovupoužiteľnosť kódu. HTML dokument by nemal obsahovať veľa inline štýlov ani vložený JavaScript priamo v atribútoch typu `onclick`. Lepšie je používať externé súbory:

```
<link rel="stylesheet" href="style.css">
<script src="app.js"></script>
```

Nesprávny prístup:

```
<button onclick="alert('Ahoj')" style="color:red">Klikni</button>
```

Lepší prístup:

```
<button id="helloButton" class="important">Klikni</button>
```

```
.important {
    color: red;
}
```

```
document.getElementById("helloButton")
    .addEventListener("click", function () {
        alert("Ahoj");
    });
```

1.8 Šablóny

Šablóny slúžia na oddelenie dát od výsledného HTML výstupu. Namiesto toho, aby sa HTML skladalo ručne reťazcami, použije sa šablóna, do ktorej sa dosadia dáta.

Príklad jednoduchkej šablóny:

```
<h1>{{title}}</h1>
<p>{{content}}</p>
```

Po dosadení dát môže vzniknúť:

```
<h1>Novinka</h1>
<p>Dnes bola spustená nová verzia aplikácie.</p>
```

Šablóny sa používajú:

- na serveri, napríklad pri generovaní HTML z databázy,
- na klientovi, napríklad vo frameworkoch a knižniciach,
- pri komponentovom vývoji webových aplikácií.

Výhody šablón:

- lepšie oddelenie logiky a prezentácie,
- opakovateľné použitie komponentov,
- jednoduchšia údržba,
- prehľadnejší kód.

1.9 JavaScript transpilery

Transpiler je source-to-source kompilátor. To znamená, že prekladá zdrojový kód z jednej verzie alebo dialektu jazyka do inej verzie toho istého alebo podobného jazyka.

V kontexte JavaScriptu sa transpiler používa napríklad na preklad moderného JavaScriptu do staršej verzie JavaScriptu, ktorú podporujú staršie prehliadače. Typickým príkladom je preklad ECMAScript 2015+ do staršieho JavaScriptu.

Príklady JavaScript transpilerov:

- Babel,
- Traceur.

Transpilery umožňujú používať moderné jazykové vlastnosti, napríklad:

- `let` a `const`,
- arrow funkcie,

- triednu syntax,
- moduly,
- JSX.

Príklad moderného JavaScriptu:

```
const add = (a, b) => a + b;
```

Po transpile môže vzniknúť starší zápis:

```
var add = function (a, b) {
  return a + b;
};
```

Výhody transpilerov:

- možnosť používať moderný JavaScript,
- lepšia kompatibilita so staršími prehliadačmi,
- podpora jazykových rozšírení,
- čitateľnejší a modernejší zdrojový kód.

Nevýhody:

- komplikovanejší build proces,
- potreba nástrojov ako bundler alebo package manager,
- výsledný kód môže byť menej čitateľný,
- niektoré vlastnosti vyžadujú polyfilly alebo runtime podporu.

1.10 jQuery

jQuery je JavaScriptová knižnica, ktorej cieľom je zjednodušiť bežné úlohy pri tvorbe webových stránok. Zjednodušuje najmä:

- vyhľadávanie prvkov v DOM,
- manipuláciu s HTML dokumentom,
- obsluhu udalostí,
- animácie,
- AJAX komunikáciu,
- rozdiely medzi prehliadačmi.

Príklad použitia jQuery:

```
$("#message").text("Ahoj svet!");
```

To isté v čistom JavaScripte:

```
document.getElementById("message").textContent = "Ahoj svet!";
```

1.10.1 Výhody jQuery

Medzi hlavné výhody jQuery patria:

- jednoduchá syntax,
- kratší kód,
- zjednodušená práca s DOM,
- jednoduchá obsluha udalostí,
- jednoduchšia AJAX komunikácia,
- veľké množstvo pluginov,
- historicky dobrá podpora rôznych prehliadačov.

Príklad obsluhy udalosti:

```
$("#button").click(function () {  
    alert("Kliknuté");  
});
```

Príklad AJAX požiadavky:

```
$.ajax({  
    url: "script.php",  
    type: "POST",  
    data: { name: "John" }  
}).done(function (msg) {  
    $("#result").html(msg);  
});
```

1.10.2 Nevýhody jQuery

Nevýhody jQuery:

- pridáva ďalšiu závislosť do projektu,
- zväčšuje veľkosť stránky,
- mnohé jeho funkcie dnes nahrádza natívne DOM API,
- natívne selektory ako `querySelectorAll` môžu byť rýchlejšie,
- pri moderných frameworkoch býva často nepotrebné,
- môže viesť k menej štruktúrovanému kódu, ak sa používa nekontrolovane.

Dnes platí, že jQuery je užitočné najmä pri starších projektoch, pri jednoduchej manipulácii s DOM alebo pri potrebe použiť existujúce plugíny. V moderných aplikáciách ho často nahrádzajú natívne API alebo frameworky ako React, Vue či Angular.

1.11 Prechod od statických stránok k dynamickým aplikáciám

1.11.1 Statické stránky

Statická stránka je uložený HTML súbor, ktorý server odošle klientovi bez výrazného spracovania. Každý používateľ dostane rovnaký obsah.

Typický príklad:

GET /index.html

Server vráti súbor `index.html`. Obsah stránky sa nemení podľa používateľa ani podľa databázy.

Výhody statických stránok:

- jednoduchosť,
- rýchlosť,
- nízke nároky na server,
- jednoduché nasadenie.

Nevýhody:

- slabá interaktivita,
- nemožnosť personalizácie,
- komplikovaná údržba väčšieho množstva stránok,
- chýba práca s používateľskými účtami a databázou.

1.11.2 Dynamické stránky

Dynamická stránka sa generuje na základe požiadavky, používateľa, databázy alebo iného stavu aplikácie. Server môže napríklad z databázy načítať články, produkty alebo používateľský profil a vygenerovať HTML.

Príklad:

GET /article?id=10

Server načíta článok s identifikátorom 10 z databázy a vygeneruje HTML odpoveď.

Dynamický obsah môže byť generovaný:

- na serveri,
- na klientovi pomocou JavaScriptu,
- kombináciou oboch prístupov.

1.12 Cookies

Cookie je malý údaj uložený v prehliadači používateľa. Server ho môže poslať v HTTP odpovedi a prehliadač ho následne posiela späť pri ďalších požiadavkách na rovnakú doménu.

Cookies sa používajú napríklad na:

- identifikáciu session,
- zapamätanie nastavení používateľa,
- ukladanie stavu prihlásenia,
- analytiku alebo personalizáciu.

Príklad HTTP hlavičky:

Set-Cookie: sessionId=abc123; HttpOnly; Secure

Pri ďalšej požiadavke prehliadač pošle:

Cookie: sessionId=abc123

Dôležité bezpečnostné atribúty cookies:

- **HttpOnly** – cookie nie je dostupná cez JavaScript,
- **Secure** – cookie sa posiela iba cez HTTPS,
- **SameSite** – obmedzuje posielanie cookie pri požiadavkách z iných stránok.

1.13 Sessions

Session je mechanizmus na uchovanie stavu používateľa na serveri. Keďže HTTP je bezstavové, session umožňuje rozpoznať, že viacero požiadaviek patrí tomu istému používateľovi.

Typický princíp:

1. Používateľ sa prihlási.
2. Server vytvorí session a uloží ju na serveri.
3. Server pošle klientovi cookie so session identifikátorom.
4. Pri ďalších požiadavkách klient posiela session ID.
5. Server podľa session ID nájde údaje používateľa.

Session sa používa napríklad na:

- prihlásenie používateľa,
- nákupný košík,

- dočasné nastavenia,
- ochranu prístupu k súkromným častiam aplikácie.

Rozdiel medzi cookie a session je v tom, že cookie je uložená na klientovi, zatiaľ čo session dáta sú typicky uložené na serveri. Cookie často obsahuje iba identifikátor session.

1.14 Upload súborov

Upload znamená odosielanie súboru z klienta na server. V HTML sa používa formulár s polom typu file a kódovaním multipart/form-data.

Príklad:

```
<form action="/upload" method="post" enctype="multipart/form-data">
  <input type="file" name="file">
  <button type="submit">Nahrať</button>
</form>
```

Na strane servera treba:

- skontrolovať veľkosť súboru,
- overiť typ súboru,
- bezpečne uložiť súbor,
- zabrániť prepísaniu existujúcich súborov,
- chrániť aplikáciu pred škodlivými súbormi.

Upload sa používa napríklad pri profilových obrázkoch, dokumentoch, galériách alebo importoch dát.

1.15 Dynamické generovanie obsahu a databáza

Dynamické aplikácie často používajú databázu. Databáza uchováva používateľov, články, komentáre, objednávky, produkty alebo iné dáta.

Typický tok požiadavky:

1. Klient pošle HTTP požiadavku.
2. Server spracuje parametre požiadavky.
3. Server vykoná databázový dotaz.
4. Výsledky vloží do šablóny.
5. Server odošle HTML alebo JSON odpoveď.

Pri serverovo generovanom HTML server vráti už hotovú stránku. Pri klientsky generovanom obsahu server často vráti len dáta vo formáte JSON a JavaScript na klientovi z nich vytvorí výsledné používateľské rozhranie.

Príklad JSON odpovede:

```
{
  "id": 10,
  "title": "Nový článok",
  "author": "Admin"
}
```

1.16 AJAX a dynamické aplikácie

AJAX znamená Asynchronous JavaScript and XML. V praxi dnes často nejde o XML, ale o JSON. AJAX umožňuje posilať požiadavky na server bez obnovenia celej stránky.

Výhody AJAXu:

- stránka sa nemusí celá reloadovať,
- používateľské rozhranie je plynulejšie,
- dá sa načítať iba potrebná časť dát,
- vhodné pre validáciu formulárov, chaty, administrácie a hry.

Príklad pomocou jQuery:

```
$.ajax({
  url: "data.php",
  type: "GET",
  dataType: "json"
}).done(function (data) {
  console.log(data);
});
```

1.17 WebSockets

Pri klasickom HTTP klient vždy začína komunikáciu požiadavkou. Pre aplikácie, ktoré potrebujú okamžité obojsmerné spojenie, napríklad chat alebo online hru, je vhodný WebSocket.

Pred WebSockets sa používali napríklad:

- HTTP polling,
- long polling,
- HTTP streaming.

WebSocket vytvorí trvalé obojsmerné spojenie medzi klientom a serverom. Po otvorení spojenia môžu obe strany posilať správy.

Príklad:

```
var ws = new WebSocket("ws://example.com/socket");

ws.onopen = function () {
    ws.send("Hello");
};

ws.onmessage = function (event) {
    console.log(event.data);
};
```

WebSockets znižujú latenciu a réžiu komunikácie pri aplikáciách, ktoré vyžadujú častú výmenu správ.

1.18 Zhrnutie

Základ internetových aplikácií tvorí spolupráca technológií HTTP, HTML, CSS, SVG a JavaScript. HTML definuje obsah, CSS vzhľad a JavaScript správanie. SVG umožňuje vektorovú grafiku priamo manipulovateľnú cez DOM. Oddelenie obsahu, prezentácie a kódu zlepšuje udržiavateľnosť aplikácie. Šablóny umožňujú oddeliť dáta od výsledného HTML výstupu.

Transpilery, napríklad Babel a Traceur, umožňujú používať moderný JavaScript aj v prostrediach, ktoré ho natívne nepodporujú. jQuery historicky výrazne zjednodušilo prácu s DOM, udalosťami, animáciami a AJAXom, ale v moderných aplikáciách je často nahrádzané natívnymi API alebo frameworkmi.

Prechod od statických stránok k dynamickým aplikáciám zahŕňa používanie cookies, sessions, uploadu súborov, databáz, šablón, AJAX komunikácie a prípadne WebSockets. Výsledkom sú aplikácie, ktoré reagujú na používateľa, pracujú s dátami a poskytujú personalizovaný obsah.

2 JavaScript OOP

2.1 Objektovo orientované programovanie v JavaScripte

JavaScript podporuje objektovo orientované programovanie, ale na rozdiel od klasických jazykov ako Java alebo C++ je založený na **prototypoch**. To znamená, že objekty nededia primárne od tried, ale priamo od iných objektov.

V triedne orientovaných jazykoch platí:

- trieda je samostatná entita a opisuje dátový typ,
- objekt je inštanciou triedy,
- trieda definuje množinu atribútov a metód,

- dedenie prebieha cez hierarchiu tried,
- štruktúra objektu je typicky určená triedou.

V JavaScripte platí:

- historicky neexistovali triedy v rovnakom zmysle ako v Jave,
- objekty sú v podstate slovníky vlastností,
- vlastnosť má meno a hodnotu,
- hodnota vlastnosti môže byť ľubovoľná JavaScriptová hodnota,
- objekty môžu dynamicky získavať alebo strácať vlastnosti,
- dedenie prebieha cez prototypový reťazec.

Objekt v JavaScripte možno chápať ako kontajner vlastností so skrytým odkazom na prototypový objekt. Ak sa požadovaná vlastnosť nenachádza priamo v objekte, JavaScript ju hľadá v jeho prototypu, potom v prototypu prototypu atď.

2.2 Rozdiel medzi triednym a prototypovým OOP

V triednom OOP je základom **trieda**. Trieda opisuje, aké atribúty a metódy budú mať jej inštancie. Objekt sa vytvára ako inštancia triedy.

Príklad v triednom štýle:

```
class Person {
    String name;

    void sayHello() {
        System.out.println(name);
    }
}
```

V prototypovom OOP je základom **objekt**. Nový objekt môže dediť vlastnosti priamo z iného objektu.

Príklad v JavaScripte:

```
var person = {
    name: "Janko",
    sayHello: function () {
        console.log(this.name);
    }
};

var student = Object.create(person);
student.name = "Fero";
student.sayHello(); // Fero
```

Objekt `student` nemá vlastnú metódu `sayHello`. Tá sa nájde v jeho prototype, ktorým je objekt `person`.

2.3 Prototypový reťazec

Každý objekt má interný odkaz na svoj prototyp. V staršej terminológii sa tento odkaz často označuje ako `__proto__`. Pri prístupe k vlastnosti JavaScript postupuje takto:

1. Najprv hľadá vlastnosť priamo v objekte.
2. Ak ju nenájde, hľadá ju v prototype objektu.
3. Ak ju nenájde ani tam, pokračuje v prototype prototypu.
4. Reťazec končí pri `Object.prototype`.
5. Ak sa vlastnosť nenájde, výsledkom je `undefined`.

Príklad:

```
var animal = {
  eats: true
};

var rabbit = Object.create(animal);
rabbit.jumps = true;

console.log(rabbit.jumps); // true
console.log(rabbit.eats);  // true
```

Vlastnosť `jumps` je vlastná vlastnosť objektu `rabbit`. Vlastnosť `eats` sa nájde v prototype `animal`.

2.4 `__proto__` a `prototype`

V JavaScripte je dôležité rozlišovať medzi položkami `__proto__` a `prototype`.

2.4.1 `__proto__`

Položka `__proto__` označuje prototyp konkrétneho objektu. Je to objekt, ktorý sa používa pri vyhľadávaní vlastností v prototypovom reťazci.

Príklad:

```
var parent = {
  x: 10
};

var child = Object.create(parent);
```

```
console.log(child.__proto__ === parent); // true
console.log(child.x);                    // 10
```

Objekt `child` nemá vlastnú vlastnosť `x`. JavaScript ju preto nájde v objekte `parent`, ktorý je jeho prototypom.

2.4.2 prototype

Položka `prototype` sa nachádza na funkciách, ktoré môžu byť použité ako konštruktory. Keď vytvoríme objekt pomocou operátora `new`, JavaScript nastaví `__proto__` nového objektu na hodnotu `prototype` konštruktorovej funkcie.

Príklad:

```
function Person(name) {
    this.name = name;
}

Person.prototype.sayHello = function () {
    console.log("Ahoj, volam sa " + this.name);
};

var p = new Person("Janko");

p.sayHello(); // Ahoj, volam sa Janko
console.log(p.__proto__ === Person.prototype); // true
```

Tu platí:

- `Person.prototype` je objekt, ktorý bude prototypom nových objektov vytvorených cez `new Person(...)`.
- `p.__proto__` je skutočný prototyp konkrétneho objektu `p`.
- `p.__proto__ === Person.prototype`.

2.4.3 Zhrnutie rozdielu

prototype	__proto__
Vlastnosť konštruktorovej funkcie	Vlastnosť konkrétneho objektu
Používa sa pri vytváraní objektu cez <code>new</code>	Používa sa pri vyhľadávaní vlastností
Určuje budúci prototyp inštancií	Odkazuje na aktuálny prototyp objektu
Napr. <code>Person.prototype</code>	Napr. <code>p.__proto__</code>

Zmena `__proto__` existujúceho objektu sa neodporúča, pretože je pomalá a môže zhoršiť optimalizácie JavaScriptového enginu. Lepšie je vytvárať objekty s požadovaným prototypom už od začiatku, napríklad pomocou `Object.create`.

2.5 Konštruktorové funkcie

Pred ES6 sa na vytváranie objektov často používali konštruktorové funkcie.

```
function Car(brand) {  
    this.brand = brand;  
}  
  
Car.prototype.drive = function () {  
    console.log(this.brand + " ide.");  
};  
  
var car = new Car("Skoda");  
car.drive();
```

Operátor **new** vykoná tieto kroky:

1. vytvorí nový objekt,
2. nastaví jeho prototyp na `Car.prototype`,
3. nastaví **this** v konšuktore na nový objekt,
4. vykoná telo konšuktora,
5. vráti nový objekt, ak konštruktor explicitne nevráti iný objekt.

2.6 Triedna syntax v ECMAScript 2015

Moderný JavaScript zaviedol syntax **class**. Táto syntax však nemení podstatu dedenia v JavaScripte. Je to najmä pohodlnejší zápis nad prototypovým modelom.

Príklad:

```
class User {  
    constructor(name) {  
        this.name = name;  
    }  
  
    sayHi() {  
        console.log(this.name);  
    }  
}  
  
let user = new User("John");  
user.sayHi();
```

Tento zápis je podobný konštruktorovej funkcii:


```
function User(name) {
    this.name = name;
}

User.prototype.sayHi = function () {
    console.log(this.name);
};
```

Aj pri použití `class` teda metódy končia na prototype a objekty stále dedia cez prototypový reťazec.

2.7 Význam `this` v JavaScripte

Kľúčové slovo `this` označuje objekt, v ktorého kontexte bola funkcia zavolaná. Jeho hodnota nezávisí primárne od miesta, kde je funkcia definovaná, ale od spôsobu, akým je zavolaná.

2.7.1 Volanie ako metóda objektu

```
var obj = {
    name: "Janko",
    print: function () {
        console.log(this.name);
    }
};

obj.print(); // Janko
```

Pri volaní `obj.print()` je `this` nastavené na objekt `obj`.

2.7.2 Samostatné volanie funkcie

```
var f = obj.print;
f();
```

V tomto prípade už funkcia nie je volaná ako metóda objektu `obj`. Hodnota `this` preto môže byť globálny objekt, alebo v strict mode `undefined`.

2.8 Manipulácia s `this` pomocou `call`

Metóda `call` umožňuje zavolať funkciu s explicitne nastavenou hodnotou `this`. Argumenty sa odovzdávajú jednotlivo.

Syntax:

```
funkcia.call(thisArg, arg1, arg2, ...)
```

Príklad:

```
function introduce(greeting) {
    console.log(greeting + ", som " + this.name);
}
```

```
var person = { name: "Janko" };
```

```
introduce.call(person, "Ahoj"); // Ahoj, som Janko
```

Funkcia `introduce` nie je metódou objektu `person`, ale pomocou `call` ju zavoláme tak, akoby bola vykonaná v kontexte tohto objektu.

2.9 Manipulácia s `this` pomocou `apply`

Metóda `apply` je podobná ako `call`, ale argumenty sa odovzdávajú ako pole.

Syntax:

```
funkcia.apply(thisArg, [arg1, arg2, ...])
```

Príklad:

```
function introduce(greeting, punctuation) {
    console.log(greeting + ", som " + this.name + punctuation);
}
```

```
var person = { name: "Janko" };
```

```
introduce.apply(person, ["Ahoj", "!"]); // Ahoj, som Janko!
```

`apply` je užitočné vtedy, keď máme argumenty pripravené v poli alebo v objektu podobnom poli.

Príklad použitia:

```
var numbers = [4, 2, 9, 1];
```

```
var max = Math.max.apply(null, numbers);
console.log(max); // 9
```

2.10 Manipulácia s `this` pomocou `bind`

Metóda `bind` nevykoná funkciu hneď. Vytvorí novú funkciu, v ktorej je `this` natrvalo naviazané na zadaný objekt.

Syntax:

```
var novaFunkcia = funkcia.bind(thisArg, arg1, arg2, ...)
```

Príklad:

```

var person = {
  name: "Janko"
};

function printName() {
  console.log(this.name);
}

```

```

var boundPrintName = printName.bind(person);
boundPrintName(); // Janko

```

bind sa často používa pri callbackoch, kde by sa inak stratila hodnota `this`.
Príklad problému:

```

function Counter() {
  this.value = 0;

  setInterval(function () {
    this.value++;
    console.log(this.value);
  }, 1000);
}

```

Vo vnútornej anonymnej funkcii nemusí `this` ukazovať na objekt `Counter`.
Riešenie pomocou `bind`:

```

function Counter() {
  this.value = 0;

  setInterval(function () {
    this.value++;
    console.log(this.value);
  }.bind(this), 1000);
}

```

Ďalším riešením je uložiť si `this` do premennej:

```

function Counter() {
  var self = this;
  self.value = 0;

  setInterval(function () {
    self.value++;
    console.log(self.value);
  }, 1000);
}

```

V modernom JavaScripte sa často používa arrow funkcia, pretože arrow funkcie nemajú vlastné `this`; používajú `this` z okolitého lexikálneho kontextu.

```
function Counter() {
    this.value = 0;

    setInterval(() => {
        this.value++;
        console.log(this.value);
    }, 1000);
}
```

Arrow funkcie však nie sú vhodné ako metódy objektov, pretože nemajú vlastné `this`.

2.11 Využitie call, apply a bind

Metódy `call`, `apply` a `bind` sa používajú najmä na:

- explicitné nastavenie hodnoty `this`,
- znovupoužitie metódy jedného objektu na inom objekte,
- prácu s funkciami vyššieho rádu a callbackmi,
- čiastočnú aplikáciu argumentov,
- riešenie problému strateného `this` v anonymných funkciách,
- volanie funkcie s argumentmi uloženými v poli.

Rozdiel medzi nimi:

- `call` funkciu hneď zavolá a argumenty sa odovzdávajú jednotlivo,
- `apply` funkciu hneď zavolá a argumenty sa odovzdávajú ako pole,
- `bind` funkciu hneď nezavolá, ale vráti novú funkciu s naviazaným `this`.

2.12 Privátne položky a metódy

JavaScriptové objekty historicky nemali klasické privátne atribúty a metódy ako napríklad Java. Vlastnosti objektu sú bežne verejné a dajú sa čítať alebo meniť zvonka.

Príklad verejnej vlastnosti:

```
function Account(balance) {
    this.balance = balance;
}
```

```
var acc = new Account(100);
acc.balance = 1000000; // zvonka možno zmeniť hodnotu
```

Ak chceme dosiahnuť zapuzdrenie, môžeme použiť **closures**.

2.13 Closures

Closure vzniká vtedy, keď vnútorná funkcia pristupuje k premenným vonkajšej funkcie aj po tom, čo vonkajšia funkcia skončila.

Príklad:

```
function makeAdder(x) {  
    return function (y) {  
        return x + y;  
    };  
}  
  
var add10 = makeAdder(10);  
console.log(add10(7)); // 17
```

Vnútorná funkcia si pamätá premennú `x`, hoci volanie `makeAdder(10)` už skončilo. Toto je closure.

2.14 Privátne dáta pomocou closure

Privátne položky možno vytvoriť ako lokálne premenné konštruktorovej funkcie. Tieto premenné nie sú prístupné zvonka, ale vnútorné metódy k nim majú prístup.

Príklad:

```
function Account(initialBalance) {  
    var balance = initialBalance;  
  
    this.deposit = function (amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    };  
  
    this.getBalance = function () {  
        return balance;  
    };  
}  
  
var acc = new Account(100);  
acc.deposit(50);  
  
console.log(acc.getBalance()); // 150  
console.log(acc.balance);      // undefined
```

Premenná `balance` je lokálna premenná konšuktora. Zvonka sa k nej nedá prístupiť pomocou `acc.balance`. Metódy `deposit` a `getBalance` však majú k nej prístup, pretože nad ňou vytvárajú closure.

2.15 Privátne metódy pomocou closure

Rovnakým spôsobom možno vytvoriť aj privátne metódy.

```
function Container(value) {
    var counter = 2;

    function decreaseCounter() {
        if (counter > 0) {
            counter--;
            return true;
        }
        return false;
    }

    this.getValue = function () {
        if (decreaseCounter()) {
            return value;
        }
        return null;
    };
}

var c = new Container(42);

console.log(c.getValue()); // 42
console.log(c.getValue()); // 42
console.log(c.getValue()); // null
```

Premenná `counter` aj funkcia `decreaseCounter` sú privátne. Nie sú vlastnosťami objektu `c`. Verejná je iba metóda `getValue`, ktorá k nim má prístup cez closure.

2.16 Výhody privátnych položiek cez closures

Výhody:

- skutočné zapuzdrenie dát,
- nemožnosť priamo meniť vnútorný stav objektu zvonka,
- kontrolovaný prístup cez verejné metódy,
- vhodné pre moduly a objekty s vnútorným stavom.

Nevýhody:

- metódy vytvorené v konštruktoze vznikajú pre každú inštanciu zvlášť,
- nemožno ich jednoducho zdieľať cez prototype,

- môže to byť pamäťovo náročnejšie,
- pri väčších objektoch môže byť kód menej prehľadný.

Ak metódu definujeme na prototype, zdieľajú ju všetky inštalácie:

```
Person.prototype.sayHello = function () {
    console.log(this.name);
};
```

Takáto metóda však nemá prístup k lokálnym premenným konštruktora, ak tie nie sú dostupné cez closure. Preto je medzi zdieľaním metód cez prototype a privátnymi dátami cez closure určitý kompromis.

2.17 Modulový vzor

Closures sa často používajú aj v modulovom vzore. Modul môže mať privátne premenné a navonok vráti iba verejné rozhranie.

```
var counterModule = (function () {
    var counter = 0;

    function log() {
        console.log("counter = " + counter);
    }

    return {
        increment: function () {
            counter++;
            log();
        },
        get: function () {
            return counter;
        }
    };
})();

counterModule.increment();
console.log(counterModule.get());
console.log(counterModule.counter); // undefined
```

Premenná `counter` a funkcia `log` sú privátne. Verejné sú iba metódy `increment` a `get`.

2.18 Zhrnutie

JavaScript používa prototypové objektovo orientované programovanie. Objekty dedia priamo od iných objektov cez prototypový reťazec. Na rozdiel od triedneho OOP nie je základom trieda, ale objekt a jeho prototyp.

Položka `prototype` patrí konštruktorovej funkcii a určuje, aký prototyp dostanú objekty vytvorené pomocou `new`. Položka `__proto__` patrí konkrétnemu objektu a ukazuje na jeho skutočný prototyp používaný pri vyhľadávaní vlastností.

Hodnota `this` závisí od spôsobu volania funkcie. Pomocou `call`, `apply` a `bind` možno `this` explicitne nastaviť. `call` a `apply` funkciu hneď zavolajú, zatiaľ čo `bind` vytvorí novú funkciu s naviazaným `this`.

Privátne položky a metódy možno v JavaScripte dosiahnuť pomocou closures. Lokálne premenné konšuktora nie sú prístupné zvonka, ale vnútorné funkcie si ich pamätajú a môžu s nimi pracovať. Tento mechanizmus umožňuje zapuzdrenie a tvorbu objektov s kontrolovaným verejným rozhraním.

3 JavaScript

3.1 Typované polia

Bežné JavaScriptové pole `Array` je dynamické a môže obsahovať hodnoty rôznych typov. To je pohodlné, ale nie vždy vhodné pri práci s binárnymi dátami, grafikou, zvukom, sieťovou komunikáciou alebo WebGL. Na tieto účely JavaScript poskytuje **typované polia**.

Typované polia sú rozdelené na dve základné časti:

- **buffer** – surový blok pamäte,
- **view** – pohľad na buffer, ktorý určuje typ prvkov, počiatočný offset a počet prvkov.

3.2 ArrayBuffer

`ArrayBuffer` predstavuje blok binárnych dát. Sám o sebe neurčuje, aké typy hodnôt sú v ňom uložené. Je to len súvislá oblasť pamäte v bajtoch. Nemá mechanizmus na priamy prístup k prvkom, preto sa k nemu prístupuje cez niektorý typ pohľadu.

Príklad:

```
var buf = new ArrayBuffer(16);  
console.log(buf.byteLength); // 16
```

Týmto sa vytvorí buffer veľkosti 16 bajtov. Všetky bajty sú inicializované na nulu.

3.3 View

View, teda pohľad, interpretuje buffer ako pole hodnôt určitého typu. Napríklad `Int32Array` interpretuje buffer ako pole 32-bitových celých čísel, `Uint8Array` ako pole 8-bitových neznamienkových celých čísel a podobne.

Príklad:


```
var buf = new ArrayBuffer(16);
var int32View = new Int32Array(buf);

for (var i = 0; i < int32View.length; i++) {
    int32View[i] = 2 * i;
}
```

```
console.log(int32View); // [0, 2, 4, 6]
```

Buffer má 16 bajtov. Keďže jeden prvok typu `Int32` má 4 bajty, `Int32Array` nad týmto bufferom obsahuje 4 prvky.

Ten istý buffer môže mať viacero pohľadov:

```
var buf = new ArrayBuffer(16);

var int32View = new Int32Array(buf);
var int16View = new Int16Array(buf);
var uint8View = new Uint8Array(buf);
```

Všetky tieto pohľady pracujú nad rovnakou pamäťou, ale interpretujú ju iným spôsobom.

3.4 Offset a dĺžka view

Pri vytváraní view možno zadať aj offset a dĺžku:

```
var buf = new ArrayBuffer(16);
var int16View = new Int16Array(buf, 2, 6);
```

Druhý parameter je offset v bajtoch a tretí parameter je počet prvkov. Pri typovaných poliach musí byť offset zarovnaný podľa veľkosti prvku. Napríklad pri `Int16Array` musí byť offset deliteľný dvomi.

3.5 Inicializácia typovaných polí

Typované pole možno vytvoriť aj priamo z poľa hodnôt:

```
var byteArray = new Uint8Array([1, 2, 3, 4]);
console.log(byteArray); // [1, 2, 3, 4]
```

Podkladový buffer možno získať cez vlastnosť `buffer`:

```
var buf = byteArray.buffer;
```

Konverzia typovaného poľa na bežné pole:

```
var normalArray = Array.apply([], byteArray);
```

3.6 DataView

DataView poskytuje nízkoúrovňové rozhranie na čítanie a zápis hodnôt rôznych typov z **ArrayBuffer**. Na rozdiel od typovaných polí umožňuje čítať aj nezarovnané dáta a explicitne určovať poradie bajtov, teda endianitu.

Príklad:

```
var buf = new ArrayBuffer(16);
var view = new DataView(buf);

view.setInt16(0, 256, true); // true = little-endian
console.log(view.getInt16(0, true));
```

Metódy **DataView** zahŕňajú napríklad:

- `getInt8`, `getUint8`,
- `getInt16`, `getUint16`,
- `getInt32`, `getUint32`,
- `getFloat32`, `getFloat64`,
- príslušné metódy `set...`

DataView je vhodné použiť najmä pri práci s binárnymi formátmi, sieťovými protokolmi alebo súbormi, kde potrebujeme presnú kontrolu nad uložením dát.

3.7 Strict mode

Strict mode je prísnejší režim vykonávania JavaScriptu. Zapína sa direktívou:

```
"use strict";
```

Táto direktíva môže byť uvedená:

- na začiatku skriptu – potom platí pre celý skript,
- na začiatku funkcie – potom platí iba pre danú funkciu.

Príklad:

```
function strictFunction() {
    "use strict";
    return "strict mode";
}
```

Strict mode má viacero cieľov:

- premieňa niektoré tiché chyby na výnimky,
- zjednodušuje používanie premenných,
- umožňuje lepšiu optimalizáciu JavaScript enginom,
- zvyšuje bezpečnosť,
- zakazuje syntax, ktorá by mohla kolidovať s budúcimi verziami ECMAScriptu.

3.8 Rozdiely v strict mode

V strict mode nie je možné vytvárať globálne premenné náhodným priradením:

```
"use strict";
x = 10; // ReferenceError
```

Bez strict mode by sa tým mohla vytvoriť globálna premenná `x`, čo je častý zdroj chýb.

Strict mode tiež zakazuje príkaz `with`:

```
"use strict";
with (obj) {
    x = 10;
}
```

Príkaz `with` je problematický, pretože nie je jasné, či sa premenná hľadá v lokálnom rozsahu, globálnom rozsahu alebo ako vlastnosť objektu.

Ďalší rozdiel je, že priradenie do nezapisovateľnej vlastnosti spôsobí chybu:

```
"use strict";

var o = {};
Object.defineProperty(o, "x", {
    value: 0,
    writable: false
});

o.x = 1; // TypeError
```

Strict mode mení aj správanie `this`. Pri obyčajnom volaní funkcie nie je `this` automaticky globálny objekt, ale `undefined`:

```
function f() {
    "use strict";
    return this;
}

console.log(f()); // undefined
```

3.9 Funkcie s premenlivým počtom argumentov

JavaScript nekontroluje počet argumentov podľa počtu formálnych parametrov. Funkciu možno zavolať s menším, rovnakým alebo väčším počtom argumentov.

Príklad:

```
function sum(a, b) {
    return a + b;
}
```

```
sum(1, 2);    // 3
sum("a");    // "aundefined"
sum();       // NaN
```

Každá klasická funkcia má dostupný špeciálny objekt `arguments`. Ten obsahuje všetky argumenty, s ktorými bola funkcia zavolaná, vrátane tých, ktoré nemajú zodpovedajúci formálny parameter.

Príklad funkcie s premenlivým počtom argumentov:

```
function sum() {
    var result = 0;

    for (var i = 0; i < arguments.length; i++) {
        result += arguments[i];
    }

    return result;
}

console.log(sum(1, 9, 9, 5, 3)); // 27
```

Objekt `arguments` je podobný poľu, ale nie je to skutočné pole. Má vlastnosť `length` a indexované položky, ale nemá všetky metódy poľa, napríklad `map` alebo `filter`. Ak potrebujeme normálne pole, môžeme urobiť konverziu:

```
function f() {
    var args = Array.apply([], arguments);
    return args.join(", ");
}

console.log(f("a", 1, 3.14)); // "a, 1, 3.14"
```

V modernom JavaScripte možno namiesto `arguments` použiť rest parameter:

```
function sum(...numbers) {
    return numbers.reduce(function (a, b) {
        return a + b;
    }, 0);
}
```

Rest parameter je prehľadnejší, pretože `numbers` je skutočné pole.

3.10 Block scoping vs. function scoping

Rozsah platnosti premennej určuje, kde je premenná viditeľná a použiteľná. V JavaScripte je dôležitý rozdiel medzi `var` a `let`.

3.11 var – function scoping

Premenná deklarovaná pomocou **var** má **function scope**. To znamená, že je viditeľná v celej funkcii, v ktorej bola deklarovaná, nie iba v bloku.

Príklad:

```
function varTest() {  
    var x = 31;  
  
    if (true) {  
        var x = 71; // tá istá premenná  
        console.log(x); // 71  
    }  
  
    console.log(x); // 71  
}
```

Premenná **x** v bloku **if** je tá istá premenná ako **x** mimo bloku. Preto sa po skončení bloku jej hodnota nezmení späť na 31, ale ostane 71.

3.12 let – block scoping

Premenná deklarovaná pomocou **let** má **block scope**. To znamená, že je viditeľná iba v bloku, v ktorom bola deklarovaná.

Príklad:

```
function letTest() {  
    let x = 31;  
  
    if (true) {  
        let x = 71; // iná premenná  
        console.log(x); // 71  
    }  
  
    console.log(x); // 31  
}
```

Premenná **x** v bloku **if** je iná premenná ako **x** deklarovaná na začiatku funkcie.

3.13 var a let vo for cykle

Pri **var** zostáva premenná viditeľná aj po skončení cyklu:

```
for (var i = 0; i < 10; i++) {  
    console.log(i);  
}  
  
console.log(i); // 10
```

Pri `let` je premenná viditeľná iba v rámci cyklu:

```
for (let i = 0; i < 10; i++) {  
    console.log(i);  
}  
  
console.log(i); // ReferenceError
```

Používanie `let` je bezpečnejšie, pretože obmedzuje platnosť premennej na najmenší potrebný rozsah.

3.14 Hoisting a rozdiel medzi `var` a `let`

Deklarácie pomocou `var` sú presunuté na začiatok funkcie, čo sa nazýva **hoisting**. Premenná existuje od začiatku funkcie, ale hodnota je priradená až na mieste priradenia.

Príklad:

```
function f() {  
    console.log(x); // undefined  
    var x = 10;  
    console.log(x); // 10  
}
```

Pri `let` síce premenná tiež patrí do svojho bloku, ale pred jej deklaráciou sa nachádza v takzvanej temporal dead zone. Prístup k nej pred deklaráciou spôsobí chybu:

```
function f() {  
    console.log(x); // ReferenceError  
    let x = 10;  
}
```

3.15 `const`

S `let` súvisí aj `const`. Premenná deklarovaná pomocou `const` má tiež block scope, ale nemožno ju znovu priradiť.

```
const a = 10;  
a = 20; // TypeError
```

Pri objektoch však `const` znamená nemennosť referencie, nie nemennosť objektu:

```
const obj = { key: "value" };  
obj.key = 10; // povolené
```

3.16 Reťazenie metód

Reťazenie metód, často nazývané aj **kaskádovanie** alebo **method chaining**, je návrhový vzor, v ktorom metódy vracajú objekt, na ktorom boli zavolané. Vďaka tomu možno viacero volaní zapísať za sebou do jedného výrazu.

Základná myšlienka:

```
obj.method1()
    .method2()
    .method3();
```

Aby bolo reťazenie možné, každá metóda musí vrátiť objekt, teda typicky `this` alebo inú inštanciu.

Príklad vlastného objektu:

```
function Person() {
    this.name = "";
    this.age = 0;
}

Person.prototype.setName = function (name) {
    this.name = name;
    return this;
};

Person.prototype.setAge = function (age) {
    this.age = age;
    return this;
};

Person.prototype.print = function () {
    console.log(this.name + ", " + this.age);
    return this;
};

var p = new Person();

p.setName("Janko")
.setAge(25)
.print();
```

Metódy `setName`, `setAge` a `print` vracajú `this`, preto možno ich volania spájať.

3.17 Reťazenie v jQuery

Typickým príkladom reťazenia je jQuery:

```

$("#d1")
  .html("Hello world!")
  .css("color", "red")
  .slideUp(2000)
  .slideDown(2000)
  .fadeOut()
  .fadeIn();

```

Každá z týchto metód vráti jQuery objekt, takže na ňom možno okamžite zavolať ďalšiu metódu.

3.18 Reťazenie v D3 štýle

Podobný štýl sa používa aj v D3. Objekt môže mať getter-setter metódy, ktoré pri volaní bez argumentu vracajú hodnotu a pri volaní s argumentom nastaví hodnotu a vráti samotný objekt.

Príklad:

```

function widget(spec) {
  var instance = {};
  var description;

  instance.description = function (d) {
    if (!arguments.length) {
      return description;
    }

    description = d;
    return instance;
  };

  instance.render = function () {
    console.log(description);
    return instance;
  };

  return instance;
}

var w = widget({})
  .description("Toto je widget.")
  .render();

```

Metóda `description` funguje ako getter aj setter. Ak nemá argumenty, vracia aktuálnu hodnotu. Ak argument má, nastaví hodnotu a vráti objekt `instance`, čím umožní ďalšie reťazenie.

3.19 Výhody reťazenia metód

Výhody:

- netreba vytvárať dočasné premenné pre každý krok,
- kód je kratší,
- pri vhodnom použití je dobre čitateľný,
- prirodzene sa hodí pre konfiguráciu objektov,
- často sa používa v knižniciach ako jQuery a D3.

Nevýhody:

- príliš dlhý reťazec môže byť horšie čitateľný,
- ladenie môže byť zložitejšie,
- ak niektorá metóda nevráti správny objekt, reťazec sa preruší,
- treba jasne rozlišovať metódy, ktoré menia objekt, a metódy, ktoré iba vracajú hodnotu.

3.20 Zhrnutie

Typované polia v JavaScripte umožňujú efektívne pracovať s binárnymi dátami. **ArrayBuffer** predstavuje surový blok pamäte, **view** určuje interpretáciu tejto pamäte a **DataView** umožňuje nízkoúrovňové čítanie a zápis s podporou rôznych typov a endianity.

Strict mode zapína prísnejšie pravidlá JavaScriptu. Pomáha odhaliť chyby, zakazuje problematické konštrukcie ako **with**, zabraňuje neúmyselnému vytváraniu globálnych premenných a mení niektoré tiché chyby na výnimky.

Funkcie môžu byť volané s premenlivým počtom argumentov. V klasických funkciách sú všetky argumenty dostupné cez objekt **arguments**. V modernom JavaScripte sa často používa **rest** parameter.

Premenné deklarované pomocou **var** majú function scope, zatiaľ čo premenné deklarované pomocou **let** a **const** majú block scope. Preto je **let** bezpečnejšie pri blokoch a cykloch.

Reťazenie metód je návrhový vzor, kde metódy vracajú samotný objekt. To umožňuje zapisovať viacero operácií v jednom prehľadnom reťazci. Tento štýl je typický pre jQuery, D3 a rôzne fluent API.

4 Canvas element a Scalable Vector Graphics (SVG)

4.1 Canvas element

Element `<canvas>` je HTML element, ktorý predstavuje kresliacu plochu. Sám o sebe nič nekreslí. Na kreslenie je potrebné získať kresliaci kontext pomocou JavaScriptu, napríklad dvojrozmerný kontext `2d`.

```
<canvas id="myCanvas" width="300" height="150"></canvas>

var canvas = document.getElementById("myCanvas");
var ctx = canvas.getContext("2d");

ctx.fillStyle = "cyan";
ctx.fillRect(15, 15, 70, 70);
```

Atribúty `width` a `height` určujú skutočnú veľkosť kresliacej plochy v pixeloch. Ak sa nenastavia, predvolená veľkosť je 300 x 150 pixelov. Veľkosť možno meniť aj cez DOM vlastnosti, ale tým sa vymaže aktuálny obsah plátna. Ak sa veľkosť nastaví iba cez CSS, obsah sa len škáluje na rozmer v layoute, čo môže spôsobiť deformáciu alebo rozmazanie.

Do elementu `<canvas>` možno vložiť fallback obsah pre prehliadače, ktoré canvas nepodporujú:

```
<canvas id="clock" width="150" height="150">
  
</canvas>
```

Canvas má súradnicový systém, v ktorom štandardne platí, že jedna jednotka zodpovedá jednému pixelu. Počiatok súradnicového systému je v ľavom hornom rohu, os `x` rastie doprava a os `y` rastie nadol.

4.2 SVG

SVG, teda *Scalable Vector Graphics*, je XML formát určený na zápis vektorovej grafiky. SVG nepopisuje obrázok ako pixely, ale ako množinu geometrických objektov, napríklad čiar, kruhov, obdĺžnikov, kriviek a textu.

Príklad samostatného SVG obrázka:

```
<?xml version="1.0" standalone="no"?>
<svg xmlns="http://www.w3.org/2000/svg" version="1.1">
  <circle cx="100" cy="50" r="40"
    stroke="black" stroke-width="2" fill="red" />
</svg>
```

SVG je vektorový formát, a preto je nezávislé od rozlíšenia. Obrázok možno zväčšovať a zmenšovať bez straty kvality. SVG elementy sú zároveň DOM uzly, takže ich možno vytvárať, vyhľadávať, meniť, štylovať, animovať a pripájať na ne udalosťami pomocou JavaScriptu.

4.3 Porovnanie <canvas> a SVG

Hlavný rozdiel medzi **canvas** a SVG spočíva v tom, že canvas je bitmapová kresliaca plocha, zatiaľ čo SVG je vektorový dokument.

Canvas	SVG
Bitmapový/rastrový prístup. Výsledkom kreslenia sú pixely na plátne.	Vektorový prístup. Obrázok je opísaný elementmi ako rect , circle , path , text .
Nie je prirodzene nezávislý od rozlíšenia. Pri zväčšovaní môže byť obsah rozmazaný.	Je nezávislý od rozlíšenia. Pri zväčšení sa objekty prepočítajú a ostávajú ostré.
Nakreslené objekty nie sú samostatné DOM elementy. Po nakreslení sa stávajú súčasťou pixelovej plochy.	Každý grafický objekt je DOM uzol a možno ho individuálne meniť.
CSS ovplyvňuje samotný canvas element, napríklad okraj alebo pozadie, ale nie jednotlivé nakreslené tvary.	SVG elementy možno štýlovať pomocou prezentačných atribútov alebo CSS.
Udalosti sa viažu na celý canvas ; ak chceme zistiť kliknutie na konkrétny objekt, musíme si to vypočítať sami.	Udalosti možno viazať priamo na jednotlivé SVG elementy.
Hyperlinky nie sú prirodzenou vlastnosťou jednotlivých nakreslených tvarov.	SVG podporuje skutočné hyperlinky, napríklad cez element <a> .
Vhodný pre veľké množstvo často prekresľovaných objektov, hry, simulácie, bitmapové efekty a pixelové operácie.	Vhodné pre ikony, diagramy, schémy, grafy, mapy a interaktívne vektorové objekty.
Pri každej zmene sa často prekresľuje časť alebo celé plátno.	Pri zmene sa upravuje konkrétny DOM element.

Canvas je vhodnejší vtedy, keď potrebujeme často prekresľovať scénu, napríklad pri hrách, vizualizáciách s veľkým počtom objektov alebo pri práci s pixelmi. SVG je vhodnejšie vtedy, keď chceme mať objekty ako samostatné elementy, ktoré možno štýlovať, animovať, meniť cez DOM a pripájať na ne udalosti.

Materiály k SVG uvádzajú ako výhody nezávislosť od rozlíšenia, DOM, programovú manipuláciu tvarov, štýlov a pozícií, prácu s udalosťami a skutočné hyperlinky. Pri veľmi komplexných obrázkoch s tisíckami uzlov sa však odporúča použiť skôr canvas alebo bitmapu.

4.4 Súradnicový systém v SVG

SVG má teoreticky nekonečné plátno. Viditeľná časť tohto plátna sa nazýva **viewport**. Veľkosť viewportu sa určuje atribútmi **width** a **height** elementu

`<svg>`.

```
<svg width="200px" height="200px">
  ...
</svg>
```

Súradnice v SVG rastú podobne ako pri canvas:

- os x rastie doprava,
- os y rastie nadol.

Dôležitý je atribút `viewBox`, ktorý definuje mapovanie používateľských súradníc na viewport:

```
<svg width="200px" height="200px" viewBox="0 0 100 100">
  ...
</svg>
```

Všeobecný tvar je:

```
viewBox="minX minY width height"
```

Hodnota `viewBox` umožňuje zmeniť význam súradníc. Napríklad SVG môže mať fyzickú veľkosť 200 x 200 pixelov, ale súradnicový systém môže byť definovaný od 0 do 100. Vtedy sa jednotky automaticky preškálujú na veľkosť viewportu.

4.5 Element `<g>` – zoskupovanie

Element `<g>` slúži na zoskupovanie viacerých SVG elementov do jednej logickej skupiny. Skupine možno priradiť spoločný identifikátor, štýl, transformáciu alebo popis.

Príklad:

```
<svg width="240px" height="240px" viewBox="0 0 240 240">
  <title>Grouped Drawing</title>
  <desc>Stick-figure drawing of a house</desc>

  <g id="house" style="fill: none; stroke: black;">
    <desc>House with door</desc>
    <rect x="6" y="50" width="60" height="60" />
    <polyline points="6 50, 36 9, 66 50" />
    <polyline points="36 110, 36 80, 50 80, 50 110" />
  </g>
</svg>
```

Výhody elementu `<g>`:

- zoskupuje viacero tvarov do jedného celku,
- umožňuje spoločné štylovanie,
- umožňuje spoločnú transformáciu,
- možno mu dať `id` a neskôr ho znovu použiť,
- môže obsahovať `title` alebo `desc` pre popis a prístupnosť.

V materiáloch je ukázané zoskupenie kresby domu do elementu `<g>` s identifikátorom `house`.

4.6 Element `<use>` – znovupoužitie

Element `<use>` umožňuje znovu zobraziť už existujúci SVG element alebo skupinu elementov. Odkaz na pôvodný element sa zapisuje pomocou atribútu `xlink:href`.

Príklad:

```
<svg width="400px" height="480px" viewBox="0 0 200 240">
  <g id="house" style="fill: none; stroke: black;">
    <rect x="6" y="50" width="60" height="60" />
    <polyline points="6 50, 36 9, 66 50" />
    <polyline points="36 110, 36 80, 50 80, 50 110" />
  </g>

  <use xlink:href="#house" x="70" y="0" />
  <use xlink:href="#house" x="0" y="110" />
  <use xlink:href="#house" x="70" y="110" />
</svg>
```

Týmto sa rovnaký domček zobrazí viackrát na rôznych pozíciách. Element `<use>` teda znižuje duplicitu kódu a umožňuje vytvoriť jednu definíciu, ktorá sa použije viackrát. Materiály zdôrazňujú, že `xlink:href` určuje URI použitého elementu a že `<use>` odkazuje na už vytvorený objekt.

4.7 Element `<defs>` – definície bez zobrazenia

Element `<defs>` slúži na definovanie objektov, ktoré sa priamo nezobrazia. Zobrazia sa až vtedy, keď sa na ne odkážeme napríklad cez `<use>` alebo cez vlastnosť ako `fill: url(#id)`.

Príklad:

```
<svg width="400px" height="480px" viewBox="0 0 200 240">
  <defs>
    <g id="house" style="stroke: black;">
      <rect x="0" y="41" width="60" height="60" />
      <polyline points="0 41, 30 0, 60 41" />
    </g>
  </defs>
```

```

        <polyline points="30 101, 30 71, 44 71, 44 101" />
    </g>
</defs>

    <use xlink:href="#house" x="70" y="0" fill="red" />
    <use xlink:href="#house" x="0" y="110" fill="blue" />
    <use xlink:href="#house" x="70" y="110" fill="green" />
</svg>

```

Objekty v `<defs>` sú definované, ale nie sú samy zobrazené. SVG špecifikácia odporúča umiestňovať objekty určené na znovupoužitie do `<defs>`, pretože to môže pomôcť efektívnemu spracovaniu dokumentu.

4.8 Transformácia súradnicového systému v SVG

V SVG možno transformovať jednotlivé objekty alebo celé skupiny pomocou atribútu `transform`. Podporované sú najmä tieto transformácie:

- `translate(dx, dy)` – posunutie,
- `rotate(angle)` – rotácia,
- `scale(s)` alebo `scale(sX, sY)` – zmena mierky,
- `skewX(angle)` – skosenie v osi x,
- `skewY(angle)` – skosenie v osi y.

Uhly sa zadávajú v stupňoch.

Príklad:

```

<svg width="200px" height="200px" viewBox="0 0 200 200">
    <g id="square">
        <rect x="0" y="0" width="20" height="20"
            style="fill: black; stroke-width: 2;" />
    </g>

    <circle cx="50" cy="50" r="5" fill="red" />

    <use xlink:href="#square"
        transform="translate(50,50) rotate(45) scale(2,3)" />
</svg>

```

Transformácie sa skladajú v uvedenom poradí a výsledkom je transformovaný súradnicový systém daného elementu. Transformácie možno aplikovať na jednotlivý tvar aj na skupinu `<g>`, čím sa transformujú všetky prvky v skupine naraz.

4.9 Transformácie v canvas

Aj canvas podporuje transformácie súradnicového systému. Afinné transformácie možno reprezentovať maticou v homogénnych súradniciach. Transformácie umožňujú posunutie, škálovanie, rotáciu a skosenie. Skladanie transformácií zodpovedá násobeniu transformačných matíc.

V canvas API sa používajú napríklad metódy:

- `translate(x, y)`,
- `rotate(angle)`,
- `scale(x, y)`,
- `transform(a, b, c, d, e, f)`,
- `setTransform(a, b, c, d, e, f)`.

Na rozdiel od SVG sa však transformuje stav kresliaceho kontextu, nie samostatný DOM element. Preto sa často používa:

```
ctx.save();  
// transformácie a kreslenie  
ctx.restore();
```

4.10 Gradients

Gradient je plynulý prechod medzi farbami. Canvas aj SVG podporujú lineárne a radiálne gradienty.

4.10.1 Gradients v SVG

V SVG sa gradienty typicky definujú v elemente `<defs>` a potom sa použijú cez `url(#id)`.

Príklad lineárneho gradientu:

```
<svg width="200px" height="200px" viewBox="0 0 200 200">  
  <defs>  
    <linearGradient id="three_stops">  
      <stop offset="0%" style="stop-color: #ffcc00;" />  
      <stop offset="33.3%" style="stop-color: #cc6699" />  
      <stop offset="100%" style="stop-color: #66cc99;" />  
    </linearGradient>  
  
    <linearGradient id="diagonal"  
      xlink:href="#three_stops"  
      x1="0%" y1="0%" x2="100%" y2="100%" />  
  </defs>
```

```

<rect x="20" y="20" width="160" height="60"
      style="fill: url(#three_stops); stroke: black;" />

<rect x="20" y="120" width="160" height="60"
      style="fill: url(#diagonal); stroke: black;" />
</svg>

```

Gradient `three_stops` definuje farebné prechody pomocou elementov `<stop>`. Druhý gradient `diagonal` z neho vychádza cez `xlink:href` a mení smer gradientu.

4.10.2 Gradientsy v canvas

V canvas sa gradient vytvorí cez kresliaci kontext:

```

var grad = ctx.createLinearGradient(0, 0, 200, 0);
grad.addColorStop(0.0, "red");
grad.addColorStop(0.5, "yellow");
grad.addColorStop(1.0, "green");

```

```

ctx.fillStyle = grad;
ctx.fillRect(10, 10, 180, 50);

```

Pre radiálny gradient:

```

var grad = ctx.createRadialGradient(75, 50, 5, 90, 60, 100);
grad.addColorStop(0, "white");
grad.addColorStop(1, "black");

```

```

ctx.fillStyle = grad;
ctx.fillRect(0, 0, 200, 200);

```

Materiály uvádzajú metódy `createLinearGradient`, `createRadialGradient` a `addColorStop`, kde pozícia zastávky je číslo medzi 0.0 a 1.0.

4.11 Zarovnanie textu na cestu v SVG

SVG umožňuje umiestniť text pozdĺž ľubovoľnej cesty. Najprv sa v `<defs>` definuje cesta `<path>` s identifikátorom a potom sa v texte použije element `<textPath>`.

Príklad:

```

<svg width="500" height="500" viewBox="0 0 400 400">
  <defs>
    <path id="curvepath"
          d="M30 40 C 50 10, 70 10, 120 40
            S 150 0, 200 40"
    />
  </defs>
  <textPath href="#curvepath">
    Text on the curve
  </textPath>
</svg>

```



```

        style="stroke: gray; fill: none;" />
    </defs>

    <text style="font-size: 12;">
        <textPath xlink:href="#curvepath">
            Following a cubic Bezier curve.
        </textPath>
    </text>
</svg>

```

Element `<textPath>` spôsobí, že text sleduje tvar definovanej cesty. Cesta môže byť napríklad kubická Bézierova krivka, cesta so zaobleným rohom alebo aj zložitejšia prerušovaná cesta. V materiáloch je ukázaný text umiestnený na kubickej Bézierovej krivke, na zaoblenom rohu, na ostrom rohu aj na prerušenej ceste.

V canvas existujú textové metódy ako `fillText`, `strokeText` a vlastnosti `textAlign` či `textBaseline`, ale zarovnanie textu na ľubovoľnú cestu nie je priamo tak prirodzené ako v SVG. Pri canvas by sa muselo riešiť ručným výpočtom pozícií a rotácií znakov.

4.12 Data URL format

Data URL je spôsob, ako vložiť dáta priamo do URL. Namiesto odkazu na externý súbor obsahuje URL priamo zakódovaný obsah.

Všeobecný tvar je:

```
data:[<MIME type>][;charset=<charset>][;base64],<encoded data>
```

Časti Data URL:

- **data:** – označuje, že ide o Data URL,
- **MIME type** – typ obsahu, napríklad `image/png`,
- **charset** – znaková sada, používa sa najmä pri texte,
- **base64** – označuje, že dáta sú kódované pomocou Base64,
- **encoded data** – samotné zakódované dáta.

Ak sa MIME typ neuvedie, predvolená hodnota je `text/plain`. Ak sa nepoužije `base64`, použije sa štandardné URL kódovanie. Pri binárnych dátach, napríklad obrázkoch, je typické použiť Base64.

4.13 canvas.toDataURL()

Canvas možno exportovať ako Data URL pomocou metódy `toDataURL()`.

```
var url = canvas.toDataURL("image/png");
```

Predvolený formát je `image/png`. Možno použiť aj formáty ako `image/jpeg` alebo `image/webp`. Pri JPEG alebo WebP možno zadať aj kvalitu ako číslo medzi 0.0 a 1.0.

```
var jpeg = canvas.toDataURL("image/jpeg", 0.75);
```

Metóda vráti reťazec obsahujúci Data URL reprezentáciu aktuálneho obsahu canvasu.

4.14 Data URL a element `<img>`

Data URL možno použiť ako hodnotu atribútu `src` elementu `<img>`. Tým sa obrázok vloží priamo do HTML alebo sa dynamicky nastaví z JavaScriptu.

Príklad:

```
<img id="myImg" width="300" height="300">

var canvas = document.getElementById("myCanvas");
var img = document.getElementById("myImg");

img.src = canvas.toDataURL("image/png");
```

Tento postup sa dá použiť napríklad na zobrazenie výsledku kreslenia z canvasu ako bežného obrázka. V materiáloch je uvedený aj príklad, kde sa canvas vykreslí, skryje cez CSS a jeho obsah sa priradí do `src` elementu `<img>`.

4.15 Výhody a nevýhody Data URL

Výhody:

- netreba samostatný HTTP request na obrázok,
- dáta sú priamo súčasťou dokumentu alebo JavaScriptu,
- vhodné pre malé ikony alebo dynamicky generované obrázky,
- jednoduchý export obsahu canvasu.

Nevýhody:

- Base64 zápis zväčšuje veľkosť dát,
- veľké obrázky spôsobujú veľmi dlhé reťazce,
- horšia čitateľnosť HTML alebo CSS,
- menej vhodné pre veľké alebo často zdieľané súbory,
- externé súbory sa zvyčajne lepšie cachujú prehliadačom.

4.16 Zhrnutie

Canvas a SVG slúžia na kreslenie grafiky vo webových aplikáciách, ale ich model je odlišný. Canvas je pixelová kresliaca plocha ovládaná JavaScriptom. Je vhodný na rýchle prekresľovanie, hry, simulácie, bitmapové operácie a scény s veľkým množstvom objektov. Nakreslené prvky však nie sú DOM elementy, preto sa nedajú jednotlivo štýlovať ani jednoducho obsluhovať udalosťami.

SVG je XML vektorový formát. Je nezávislý od rozlíšenia, jeho objekty sú DOM uzly, dajú sa štýlovať cez CSS, meniť JavaScriptom, animovať a môžu mať vlastné udalosti alebo odkazy. Pri veľmi komplexných scénach s tisíckami elementov však môže byť menej efektívny.

Element `<g>` slúži na zoskupenie SVG prvkov, `<use>` na ich znovupoužitie a `<defs>` na definovanie objektov bez priameho zobrazenia. Transformácie menia súradnicový systém pomocou atribútu `transform`. Gradienty sa v SVG definujú najčastejšie v `<defs>` a používajú cez `url(#id)`. Text možno v SVG zarovnať na cestu pomocou `<textPath>`.

Data URL umožňuje vložiť dáta priamo do URL. Canvas možno exportovať do Data URL pomocou `canvas.toDataURL()` a výsledok použiť ako `src` elementu `<img>`.

5 AJAX a WebSockets

5.1 AJAX

AJAX znamená *Asynchronous JavaScript and XML*. Ide o techniku, ktorá umožňuje webovej stránke komunikovať so serverom bez toho, aby sa musela obnoviť celá stránka.

Napriek názvu AJAX dnes nemusí používať XML. Serverová odpoveď môže byť napríklad:

- obyčajný text,
- HTML fragment,
- XML dokument,
- JSON dáta.

Základná myšlienka AJAXu je:

1. používateľ vykoná akciu na stránke,
2. JavaScript vytvorí HTTP požiadavku na server,
3. server požiadavku spracuje,
4. server pošle odpoveď,
5. JavaScript odpoveď spracuje a zmení časť stránky cez DOM.

Dôležité je, že sa nemení celá stránka, ale iba jej časť. Vďaka tomu sú webové aplikácie plynulejšie a interaktívnejšie.

5.2 Použitie AJAXu

AJAX sa používa napríklad na:

- interaktívnu validáciu formulárov,
- načítanie dát bez reloadu stránky,
- odosielanie formulárov na pozadí,
- administrácie stránok,
- chat aplikácie,
- HTML5 hry,
- dynamické vyhľadávanie a filtrovanie.

Typický príklad je formulár, kde sa po zadaní mena používateľa odošle požiadavka na server, či je meno dostupné. Používateľ nemusí odoslať celý formulár ani čakať na načítanie novej stránky.

5.3 XMLHttpRequest

Základným objektom pre AJAX v staršom JavaScripte je `XMLHttpRequest`. Tento objekt umožňuje vytvoriť HTTP požiadavku, odoslať ju na server a spracovať odpoveď.

Príklad AJAX požiadavky bez jQuery:

```
var xhr;

if (window.XMLHttpRequest) {
    xhr = new XMLHttpRequest();
} else {
    // staré verzie IE
    xhr = new ActiveXObject("Microsoft.XMLHTTP");
}

xhr.onreadystatechange = function () {
    if (xhr.readyState == 4 && xhr.status == 200) {
        document.getElementById("myDiv").innerHTML =
            xhr.responseText;
    }
};

xhr.open("GET", "ajax_info.txt", true);
xhr.send();
```

V príklade sa najprv vytvorí objekt `XMLHttpRequest`. Potom sa nastaví handler `onreadystatechange`, ktorý sa volá pri zmene stavu požiadavky. Ak je `readyState == 4` a zároveň `status == 200`, znamená to, že odpoveď bola úspešne prijatá. Následne sa obsah elementu `myDiv` nahradí prijatým textom. Tento príklad je priamo uvedený v materiáloch k jQuery.

5.4 Dôležité vlastnosti XMLHttpRequest

Pri práci s `XMLHttpRequest` sú dôležité najmä:

- `open(method, url, async)` – pripraví požiadavku,
- `send(data)` – odošle požiadavku,
- `onreadystatechange` – handler zmeny stavu,
- `readyState` – stav požiadavky,
- `status` – HTTP stavový kód odpovede,
- `responseText` – textová odpoveď servera,
- `responseXML` – XML odpoveď, ak bola prijatá ako XML.

Hodnota `readyState == 4` znamená, že požiadavka je dokončená. Hodnota `status == 200` znamená úspešnú HTTP odpoveď.

5.5 AJAX s jQuery

jQuery výrazne zjednodušuje zápis AJAX požiadaviek. Namiesto priamej práce s `XMLHttpRequest` možno použiť funkciu `$.ajax`.

Príklad odoslania dát:

```
$.ajax({
  type: "POST",
  url: "save.php",
  data: {
    name: "John",
    location: "Boston"
  }
}).done(function (msg) {
  $("#div.info").text(msg);
});
```

Príklad prijatia HTML:

```
$.ajax({
  url: "test.html",
  cache: false
});
```

```
}).done(function (html) {
    $("#results").append(html);
});
```

Príklad komunikácie s ošetrením chyby:

```
var request = $.ajax({
    url: "script.php",
    type: "POST",
    data: { id: menuId },
    dataType: "html"
});

request.done(function (msg) {
    $("#log").html(msg);
});

request.fail(function (jqXHR, textStatus) {
    alert("Request failed: " + textStatus);
});
```

Materiály ukazujú, že jQuery AJAX vie odosielať dáta metódou POST, načítať HTML, načítať a spustiť JavaScript súbor a pracovať aj s XML alebo JSON dátami.

5.6 XML a JSON pri AJAXe

Pôvodne písmeno X v názve AJAX znamenalo XML. Dáta odoslané na server mohli byť zakódované ako XML, server XML spracoval a vrátil XML odpoveď.

Príklad prijatia XML:

```
$.ajax({
    url: "process.php",
    type: "POST",
    async: false,
    data: xmlDomObject,
    processData: false,
    dataType: "xml"
}).done(function (data, textStatus, xhr) {
    // data obsahuje kompletný XML dokument
});
```

V moderných aplikáciách sa však namiesto XML často používa JSON, teda *JavaScript Object Notation*. JSON je prirodzenejší pre JavaScript, je kratší a ľahko sa premieňa medzi reťazcom a objektom.

Na strane klienta:

- `JSON.stringify()` prevedie JavaScriptový objekt na JSON reťazec,

- `JSON.parse()` prevedie JSON reťazec na JavaScriptový objekt.

jQuery vie pri AJAX volaniach pracovať s JSON automaticky cez `dataType: "json"`. Materiály uvádzajú JSON ako novšiu alternatívu XML s priamou podporou v jQuery AJAX volaniach.

5.7 Výhody AJAXu

Výhody AJAXu:

- stránka sa nemusí celá obnovovať,
- používateľské rozhranie je plynulejšie,
- prenáša sa len potrebná časť dát,
- aplikácia pôsobí interaktívnejšie,
- vhodné pre validáciu formulárov,
- vhodné pre dynamické načítavanie obsahu,
- umožňuje vytvárať administrácie, chaty, hry a bohaté webové aplikácie.

AJAX bol dôležitým krokom od statických stránok k dynamickým webovým aplikáciám. Namiesto modelu “kliknutie – reload celej stránky” umožnil model, kde stránka zostáva otvorená a JavaScript len vymieňa dáta so serverom.

5.8 Nevýhody AJAXu

Nevýhody AJAXu:

- zložitejšia implementácia oproti obyčajnému odkazu alebo formuláru,
- potreba riešiť chyby siete a servera,
- potreba riešiť asynchrónnosť,
- môže skomplikovať históriu prehliadača a tlačidlo späť,
- dynamicky vložený obsah môže byť problém pre prístupnosť alebo SEO,
- pri častých požiadavkách môže zťažovať server,
- klasický AJAX nerieši skutočné server push notifikácie.

AJAX je založený na HTTP požiadavke iniciovanej klientom. Server teda bežne nemôže kedykoľvek sám od seba poslať klientovi správu. Ak chce klient zistiť, či sa na serveri niečo zmenilo, musí sa servera pýtať opakovane alebo použiť špeciálne techniky ako polling, long polling alebo streaming.

5.9 Notifikácie od servera pred WebSocketmi

Pred WebSocketmi sa na simulovanie serverových notifikácií používali najmä:

- HTTP polling,
- HTTP long polling,
- HTTP streaming.

Cieľom týchto techník bolo umožniť serveru doručiť klientovi nové dáta čo najskôr, hoci klasický HTTP model je orientovaný na požiadavky klienta.

5.10 HTTP Polling

Pri HTTP pollingu klient v pravidelných intervaloch posiela požiadavku na server a pýta sa, či existujú nové dáta.

Príklad:

```
setInterval(function () {  
    $.ajax({  
        url: "/notifications",  
        dataType: "json"  
    }).done(function (data) {  
        processNotifications(data);  
    });  
}, 5000);
```

Princíp:

1. klient každých niekoľko sekúnd pošle požiadavku,
2. server okamžite odpovie,
3. ak má nové dáta, pošle ich,
4. ak nové dáta nemá, pošle prázdnu odpoveď.

Výhody:

- jednoduchá implementácia,
- funguje nad obvyčajným HTTP,
- nepotrebuje špeciálnu podporu servera.

Nevýhody:

- veľa zbytočných požiadaviek,
- vyššia záťaž servera,
- vyššia spotreba šírky pásma,

- oneskorenie závisí od intervalu pollingu,
- nevhodné pre aplikácie vyžadujúce okamžitú reakciu.

Materiály definujú HTTP polling tak, že klient v pravidelných intervaloch posiela serveru požiadavku, aby zistil, či existujú nové informácie.

5.11 HTTP Long Polling

Long polling je vylepšenie obyčajného pollingu. Klient pošle požiadavku na server, ale server neodpovie hneď, ak nemá nové dáta. Požiadavku drží otvorenú, kým sa neobjavia dáta alebo kým neuplynie timeout.

Princíp:

1. klient pošle požiadavku,
2. server ju podrží otvorenú,
3. keď má nové dáta, odpovie,
4. klient po prijatí odpovede okamžite pošle ďalšiu požiadavku.

Výhody:

- menej prázdnych odpovedí ako pri pollingu,
- menšia latencia,
- stále používa HTTP,
- relatívne dobrá kompatibilita.

Nevýhody:

- server drží otvorené spojenia,
- klient musí po každej odpovedi vytvoriť novú požiadavku,
- stále vzniká HTTP réžia,
- zložitejšia implementácia ako obyčajný polling,
- pri veľkom počte klientov môže zatažovať server.

Materiály uvádzajú, že pri long pollingu server podrží požiadavku otvorenú, kým nemá informáciu pre klienta alebo kým nenastane timeout; potom klient požiadavku opakuje.

5.12 HTTP Streaming

Pri HTTP streaming server nikdy neoznami dokončenie HTTP odpovede. Spojenie ostáva otvorené a server do neho postupne zapisuje ďalšie dáta.

Princíp:

1. klient otvorí HTTP spojenie,
2. server začne odpoveď,
3. server odpoveď neukončí,
4. nové správy postupne posiela v rámci toho istého spojenia.

Výhody:

- nízka latencia,
- nie je nutné neustále vytvárať nové požiadavky,
- server môže dáta posilať priebežne.

Nevýhody:

- proxy servery a firewally môžu odpoveď bufferovať,
- bufferovanie zvyšuje latenciu,
- implementácia je citlivá na infraštruktúru medzi klientom a serverom,
- stále nejde o plnohodnotné obojsmerné spojenie ako pri WebSocketoch.

Materiály upozorňujú, že pri HTTP streamingu server nikdy neukončí HTTP odpoveď, spojenie zostáva otvorené, ale proxy a firewally môžu odpoveď bufferovať, čo zvyšuje latenciu doručenia správ.

5.13 WebSockets

WebSocket je technológia, ktorá umožňuje vytvoriť trvalé obojsmerné spojenie medzi klientom a serverom. Po nadviazaní spojenia môžu správy posilať obe strany bez toho, aby bolo potrebné pre každú správu vytvárať novú HTTP požiadavku.

Základný príklad:

```
var ws = new WebSocket("ws://example.com/socket");

ws.onopen = function (e) {
    console.log("Connection open...");
    ws.send("Hello WebSocket!");
};

ws.onmessage = function (e) {
```

```

        console.log("Message received:", e.data);
    };

    ws.onerror = function (e) {
        console.log("WebSocket Error:", e);
    };

    ws.onclose = function (e) {
        console.log("Connection closed");
    };

```

WebSocket spojenie začína vytvorením objektu `WebSocket`. Po odpovedi servera sa vyvolá udalosť `open`. Až vtedy je spojenie nadviazané a je bezpečné posilať správy. Materiály uvádzajú, že pred odosielaním treba čakať na `open` event alebo kontrolovať `readyState == WebSocket.OPEN`.

5.14 WebSocket API

Základné udalosti:

- `onopen` – spojenie bolo otvorené,
- `onmessage` – prišla správa,
- `onerror` – nastala chyba,
- `onclose` – spojenie bolo zatvorené.

Základné metódy:

- `send()` – odošle správu,
- `close()` – zatvorí spojenie.

WebSocket vie posilať:

- textové správy,
- `Blob`,
- `ArrayBuffer`.

Príklad poslania textovej správy:

```
ws.send("Hello WebSocket!");
```

Príklad poslania binárnych dát:

```
var a = new Uint8Array([8, 6, 7, 5, 3, 0, 9]);
ws.send(a.buffer);
```

Materiály uvádzajú aj možnosť nastaviť `binaryType` na `"blob"` alebo `"arraybuffer"` podľa toho, ako chceme prijímať binárne správy.

5.15 Stav WebSocket spojenia

Objekt `WebSocket` má vlastnosť `readyState`. Tá môže nadobúdať tieto hodnoty:

- `CONNECTING` – spojenie sa práve vytvára,
- `OPEN` – spojenie je otvorené a správy môžu prúdiť medzi klientom a serverom,
- `CLOSING` – prebieha zatvárací handshake,
- `CLOSED` – spojenie bolo zatvorené alebo sa ho nepodarilo otvoriť.

Správy sa majú posilať až v stave `OPEN`. Materiály explicitne uvádzajú, že pri `OPEN` môžu správy prúdiť medzi klientom a serverom.

5.16 Výhody WebSocketov

Výhody WebSocketov:

- vyšší výkon,
- šetrenie šírky pásma,
- nižšia záťaž CPU,
- nižšia latencia,
- jednoduché API,
- štandardizované WebSocket API a WebSocket protokol,
- možnosť stavať ďalšie protokoly nad WebSocketmi,
- súčasť moderných HTML5 webových aplikácií.

WebSokety sú efektívnejšie než polling, pretože po nadviazaní spojenia sa nemusí pre každú správu vytvárať nová HTTP požiadavka so všetkými hlavičkami. To šetrí šírku pásma aj CPU. Zároveň server nemusí čakať, kým sa klient znova opýta; môže správu poslať hneď cez otvorené spojenie.

Materiály uvádzajú ako hlavné výhody WebSocketov výkon, úsporu šírky pásma, CPU a latencie, jednoduché API, štandardy WebSocket API a WebSocket Protocol, možnosť budovať nad nimi ďalšie štandardné protokoly a súvis s HTML5.

5.17 Porovnanie AJAX a WebSocket

AJAX	WebSocket
Komunikácia je typicky založená na jednotlivých HTTP požiadavkách.	Po úvodnom handshaku vzniká trvalé obojsmerné spojenie.
Požiadavku typicky iniciuje klient.	Správy môže posilať klient aj server.
Vhodné na občasné získanie alebo odoslanie dát.	Vhodné na častú obojsmernú komunikáciu v reálnom čase.
Pre notifikácie treba polling, long polling alebo podobné techniky.	Server môže posilať notifikácie priamo cez otvorené spojenie.
Jednoduché a dobre podporované riešenie pre väčšinu bežných webových úloh.	Efektívnejšie pre chaty, hry, burzové dáta, kolaboratívne editory a realtime aplikácie.
Každá požiadavka nesie HTTP réžiu.	Po nadviazaní spojenia je réžia správ menšia.

5.18 Kedy použiť AJAX a kedy WebSocket

AJAX je vhodný, keď:

- dáta potrebujeme len občas,
- používateľ vykoná akciu a čaká odpoveď,
- ide o formulár, vyhľadávanie, filtráciu alebo načítanie časti stránky,
- nepotrebujeme okamžité serverové notifikácie.

WebSocket je vhodný, keď:

- potrebujeme komunikáciu v reálnom čase,
- server musí posilať dáta klientovi bez čakania na ďalšiu požiadavku,
- správy sa posielajú často,
- aplikácia je chat, online hra, burzový terminál, monitoring alebo kolaboratívny editor.

5.19 Zhrnutie

AJAX umožňuje posilať HTTP požiadavky zo stránky bez obnovenia celej stránky. Klasicky sa používa objekt `XMLHttpRequest`, prípadne jednoduchšie API v knižniciach ako `jQuery`. Odpoveď zo servera môže byť text, HTML, XML alebo JSON. Výhodou AJAXu je interaktivita a plynulejšie používateľské rozhranie. Nevýhodou je, že klasický AJAX je stále komunikácia iniciovaná klientom.

Pred WebSocketmi sa serverové notifikácie simulovali pomocou HTTP pollingu, long pollingu a streamingu. Polling vytvára veľa zbytočných požiadaviek, long polling drží požiadavku otvorenú do príchodu dát alebo timeoutu a streaming drží otvorenú HTTP odpoveď, ale môže trpieť bufferovaním proxy servermi a firewallmi.

WebSocket vytvára trvalé obojsmerné spojenie medzi klientom a serverom. Po otvorení spojenia môžu obe strany posilať správy. WebSokety šetria šírku pásma, CPU a latenciu, majú jednoduché API, sú štandardizované a sú súčasťou moderných HTML5 aplikácií.

6 Bezpečnosť internetových aplikácií

6.1 Úvod

Bezpečnosť internetových aplikácií sa zaoberá ochranou aplikácie, používateľov, dát a komunikácie medzi klientom a serverom. Webová aplikácia je typicky dostupná verejne cez internet, preto musí počítať s tým, že vstupy od používateľa môžu byť chybné, škodlivé alebo zámerne upravené.

Medzi základné oblasti bezpečnosti patria:

- ochrana komunikácie pomocou HTTPS,
- identifikácia a autentifikácia používateľov,
- ochrana session,
- kontrola prístupu,
- validácia vstupov,
- ochrana pred injekčnými útokmi,
- bezpečná práca s databázou,
- používanie white-list prístupu namiesto black-list prístupu.

6.2 HTTPS

HTTPS je HTTP komunikácia zabezpečená pomocou TLS. Z pohľadu aplikácie sa stále používajú HTTP požiadavky a odpovede, ale komunikácia medzi klientom a serverom je šifrovaná a autentifikovaná.

HTTPS poskytuje:

- **dôvernosť** – útočník nemôže jednoducho čítať prenášané dáta,
- **integritu** – útočník nemôže nepozorovane meniť prenášané dáta,
- **autentifikáciu servera** – klient vie overiť, že komunikuje so správnym serverom.

HTTPS je dôležité najmä pri:

- prihlasovaní,
- prenose hesiel,
- prenose session cookies,
- práci s osobnými údajmi,
- platbách,
- administrátorských rozhraniach.

Ak by aplikácia používala obyčajné HTTP, útočník na sieti by mohol odpočúvať heslá, session identifikátory alebo upravovať odpovede servera. Preto by moderná webová aplikácia mala používať HTTPS všade, nielen na prihlasovacej stránke.

6.3 Identifikácia a autentifikácia používateľov

Identifikácia znamená, že používateľ o sebe tvrdí, kto je. Napríklad zadá prihlasovacie meno alebo e-mail.

Autentifikácia znamená overenie tejto identity. Najčastejšie sa používa heslo, ale môžu sa použiť aj iné faktory.

Príklad:

- identifikácia: používateľ zadá `student@example.com`,
- autentifikácia: používateľ zadá správne heslo.

Spôsoby autentifikácie:

- heslo,
- jednorazový kód,
- hardvérový token,
- biometria,
- viacfaktorová autentifikácia,
- externý poskytovateľ identity, napríklad OAuth alebo OpenID Connect.

Heslá sa na serveri nikdy nemajú ukladať v čitateľnej podobe. Ukladať sa má iba kryptografický hash hesla so soľou. Pri úniku databázy tak útočník nezíska priamo heslá používateľov.

6.4 Autorizácia

Po autentifikácii nasleduje **autorizácia**. Autentifikácia odpovedá na otázku: Kto je používateľ? Autorizácia odpovedá na otázku: Čo tento používateľ smie robiť?

Príklad:

- bežný používateľ môže vidieť svoj profil,
- administrátor môže spravovať používateľov,
- neprihlásený používateľ nesmie pristupovať k súkromným dátam.

Autorizácia sa musí kontrolovať na serveri. Nestačí skryť tlačidlo v HTML alebo JavaScripte, pretože klientsky kód môže používateľ upraviť.

6.5 Sessions

HTTP je bezstavový protokol. Samotný protokol si nepamätá, že dve požiadavky prišli od toho istého používateľa. Preto webové aplikácie používajú session.

Typický priebeh:

1. používateľ zadá meno a heslo,
2. server overí prihlasovacie údaje,
3. server vytvorí session,
4. klientovi pošle session identifikátor, zvyčajne v cookie,
5. klient posiela session cookie pri ďalších požiadavkách,
6. server podľa session ID zistí, kto je používateľ.

Session identifikátor je citlivý údaj. Kto pozná platné session ID, môže sa vydávať za daného používateľa. Preto musí byť session ID dostatočne náhodné, dlhé a chránené.

Bezpečnostné nastavenia session cookie:

- **Secure** – cookie sa posiela iba cez HTTPS,
- **HttpOnly** – cookie nie je dostupná z JavaScriptu,
- **SameSite** – obmedzuje posielanie cookie pri požiadavkách z iných stránok,
- rozumná expirácia session,
- regenerácia session ID po prihlásení.

6.6 Session hijacking

Session hijacking je útok, pri ktorom útočník získa session identifikátor obete a následne ho použije na prístup do aplikácie ako obeť.

Možné spôsoby získania session ID:

- odpočúvanie nezabezpečenej HTTP komunikácie,
- XSS útok a čítanie cookies, ak cookie nemá `HttpOnly`,
- malware v počítači používateľa,
- únik zo servera,
- predvídateľné session identifikátory.

Ochrana:

- používať HTTPS všade,
- nastavovať cookies ako `Secure` a `HttpOnly`,
- používať náhodné a dostatočne dlhé session ID,
- regenerovať session ID po prihlásení,
- nastaviť timeout neaktivity,
- zneplatniť session po odhlásení,
- chrániť aplikáciu pred XSS.

6.7 Session riding

Session riding je iný názov pre útok typu CSRF, teda *Cross-Site Request Forgery*. Pri tomto útoku útočník zneužije fakt, že prehliadač automaticky posíla cookies k požiadavkám na danú doménu.

Používateľ je prihlásený do aplikácie. Útočník ho naláka na škodlivú stránku, ktorá spôsobí odoslanie požiadavky do legítimnej aplikácie. Prehliadač k tejto požiadavke automaticky priloží session cookie. Server si môže myslieť, že požiadavku úmyselne vykonal používateľ.

Príklad nebezpečnej požiadavky:

```

```

Ak by aplikácia vykonávala zmenu stavu cez `GET` a spoliehala sa iba na cookie, mohla by takáto požiadavka vykonať prevod.

Ochrana:

- nepoužívať `GET` na operácie meniace stav,
- používať CSRF tokeny,

- kontrolovať `Origin` alebo `Referer` hlavičku,
- používať `SameSite` cookies,
- vyžadovať opätovné potvrdenie pri kritických operáciách.

CSRF token je náhodná hodnota vložená do formulára alebo požiadavky. Útočník ju nepozná, a preto nevie vytvoriť platnú požiadavku.

6.8 URL tampering

URL tampering znamená úmyselnú manipuláciu s URL parametrami. Používateľ alebo útočník zmení hodnoty v URL a skúša, či tým získa neoprávnený prístup alebo zmení správanie aplikácie.

Príklad:

`https://example.com/profile?id=123`

Útočník zmení parameter:

`https://example.com/profile?id=124`

Ak aplikácia nekontroluje, či prihlásený používateľ smie vidieť profil s ID 124, ide o bezpečnostnú chybu.

Ďalšie príklady:

`https://shop.example/order?id=10`

`https://shop.example/order?id=11`

Alebo:

`https://example.com/download?file=invoice.pdf`

`https://example.com/download?file=../../etc/passwd`

Ochrana:

- nikdy nedôverovať URL parametrom,
- kontrolovať oprávnenia na serveri,
- validovať typ, rozsah a formát parametrov,
- nepoužívať priame názvy interných súborov,
- používať nepriame identifikátory a mapovanie na serveri,
- logovať podozrivé pokusy.

6.9 Forceful browsing

Forceful browsing je útok, pri ktorom útočník priamo zadáva URL adresy stránok alebo súborov, ku ktorým by sa bežne nedostal cez navigáciu aplikácie.

Príklad:

```
https://example.com/admin
https://example.com/admin/users
https://example.com/backup.zip
https://example.com/old-admin.php
```

Chyba nastáva vtedy, keď aplikácia skryje odkaz na administráciu, ale samotný server nekontroluje prístupové práva. Skrytie odkazu nie je bezpečnostný mechanizmus.

Ochrana:

- kontrolovať autorizáciu na každej chránenej URL,
- neponechávať na serveri zálohy, staré skripty a testovacie súbory,
- zakázať directory listing,
- používať jednotný mechanizmus kontroly prístupu,
- správne nastavovať práva k súborom a adresárom.

6.10 HTML injection

HTML injection je útok, pri ktorom útočník vloží do stránky vlastný HTML kód. Stane sa to vtedy, keď aplikácia zobrazí používateľský vstup ako HTML bez správneho escapovania.

Príklad nebezpečného vstupu:

```
<h1>Vyhral si cenu!</h1>
<a href="https://attacker.example">Klikni sem</a>
```

Ak aplikácia tento vstup vloží do stránky ako HTML, útočník môže zmeniť vzhľad stránky, vložiť falošný formulár alebo používateľa presmerovať.

V materiáloch k jQuery je ukázané, že metóda `html()` vkladá obsah ako HTML, zatiaľ čo `text()` vkladá obsah ako text. To je dôležitý rozdiel z pohľadu bezpečnosti.

Nebezpečný príklad:

```
$("#message").html(userInput);
```

Bezpečnejší príklad:

```
$("#message").text(userInput);
```

Ak používateľ zadá:

```
<b>Ahoj</b>
```

Pri `html()` sa zobrazí tučný text. Pri `text()` sa zobrazí doslovný text `<b>Ahoj</b>`.

6.11 XSS ako špeciálny prípad HTML injection

Ak útočník dokáže vložiť aj JavaScript, hovoríme o XSS, teda *Cross-Site Scripting*.

Príklad:

```
<script>alert(document.cookie)</script>
```

XSS môže umožniť:

- kradnutie session cookies,
- vykonávanie akcií v mene používateľa,
- zmenu obsahu stránky,
- phishing,
- presmerovanie používateľa.

Ochrana:

- escapovať výstup podľa kontextu,
- nepoužívať neoverený vstup v `innerHTML` alebo `.html()`,
- používať `textContent` alebo `.text()`,
- validovať vstupy,
- používať Content Security Policy,
- nastavovať cookies ako `HttpOnly`.

6.12 SQL injection

SQL injection je útok, pri ktorom útočník vloží časť SQL príkazu do vstupu aplikácie. Ak aplikácia skladá SQL dotaz spájaním reťazcov, útočník môže zmeniť význam dotazu.

Nebezpečný príklad:

```
SELECT * FROM users  
WHERE name = '$name' AND password = '$password';
```

Ak útočník zadá ako meno:

```
' OR '1'='1
```

Dotaz sa môže zmeniť na:

```
SELECT * FROM users  
WHERE name = '' OR '1'='1'  
AND password = '...';
```

Podmienka '1'='1' je vždy pravdivá, takže útočník môže obísť prihlásenie alebo získať cudzie dáta.

Iný príklad:

```
105; DROP TABLE users;
```

Ak databázový server dovoľí vykonať viac príkazov, môže dôjsť k poškodeniu databázy.

Ochrana:

- používať parametrizované dotazy,
- používať prepared statements,
- nepoužívať skladanie SQL reťazcov z používateľského vstupu,
- validovať vstupy,
- obmedziť databázové práva aplikácie,
- nezobrazovať používateľovi interné SQL chyby,
- používať ORM opatrne a správne.

Bezpečný princíp:

```
SELECT * FROM users WHERE name = ? AND password_hash = ?
```

Hodnoty sa neposielaajú ako časť SQL kódu, ale ako parametre. Databáza ich potom interpretuje ako dáta, nie ako SQL príkazy.

6.13 HTML injection vs. SQL injection

Oba útoky sú injekčné útoky, ale líšia sa cieľom:

HTML injection	SQL injection
Útočník vkladá HTML alebo JavaScript do stránky.	Útočník vkladá SQL kód do databázového dotazu.
Cieľom je prehliadač používateľa.	Cieľom je databáza.
Rizikom je zmena stránky, phishing alebo XSS.	Rizikom je únik, zmena alebo zmazanie dát.
Ochrana: escapovanie výstupu, bezpečné DOM API.	Ochrana: parametrizované dotazy a prepared statements.

Spoločný princíp ochrany je, že vstup používateľa sa nesmie chápať ako kód. Musí sa spracovať ako obyčajné dáta.

6.14 White-list vs. black-list

Pri validácii vstupov existujú dva základné prístupy:

- **black-list** – zakazujeme známe zlé hodnoty,
- **white-list** – povoľujeme iba známe dobré hodnoty.

6.14.1 Black-list

Black-list prístup sa snaží odfiltrovať nebezpečné vstupy.

Príklad:

Zakázané: `<script>`, `SELECT`, `DROP`, `--`, `../`

Problém je, že útočník môže použiť inú podobu toho istého útoku:

- iné kódovanie,
- veľké a malé písmená,
- obídenie cez medzery alebo komentáre,
- alternatívnu syntax,
- nový útok, ktorý black-list nepozná.

Black-list je preto často neúplný a ľahko obíditeľný.

6.14.2 White-list

White-list prístup povoľuje iba hodnoty, o ktorých vieme, že sú platné.

Príklad pre ID používateľa:

Povolené: celé číslo väčšie ako 0

Príklad pre triedenie:

Povolené hodnoty: `name`, `date`, `price`

Príklad:

```
if sort not in ["name", "date", "price"]:
    reject request
```

White-list je bezpečnejší, pretože nevychádza z otázky: Je tento vstup nebezpečný? ale z otázky: Je tento vstup presne jeden z povolených tvarov?

6.15 Príklady white-list validácie

E-mail:

Musí mať tvar platnej e-mailovej adresy.

Vek:

Musí byť celé číslo od 0 do 150.

Názov používateľa:

Povolené sú iba písmená, číslice, podčiarkovník, dĺžka 3 až 20 znakov.

Typ súboru pri uploade:

Povolené: image/png, image/jpeg
Zakázané implicitne: všetko ostatné.

Pri bezpečnosti je lepšie mať malé množstvo jasne povolených vstupov než sa snažiť uhádnuť všetky možné nebezpečné vstupy.

6.16 Vstupná validácia a výstupné escapovanie

Validácia vstupu a escapovanie výstupu sú odlišné veci.

Validácia vstupu kontroluje, či vstup dáva zmysel:

- vek je číslo,
- dátum má správny formát,
- ID je kladné celé číslo,
- e-mail má platný tvar.

Escapovanie výstupu zabezpečuje, že dáta sa pri výstupe neinterpretujú ako kód.

Príklad HTML escapovania:

```
< sa zmení na &lt;  
> sa zmení na &gt;  
" sa zmení na &quot;  
& sa zmení na &amp;
```

Vstup treba validovať pri prijatí a výstup escapovať pri zobrazení. Nestačí iba jedno z toho.

6.17 Zhrnutie

Bezpečnosť internetových aplikácií začína bezpečnou komunikáciou cez HTTPS. HTTPS chráni dôvernosť a integritu prenášaných dát a umožňuje overenie servera. Používateľ sa najprv identifikuje, napríklad menom alebo e-mailom, a následne autentifikuje, najčastejšie heslom. Po prihlásení sa používa session, ktorej identifikátor musí byť chránený.

Session hijacking znamená krádež session identifikátora a prevzatie relácie používateľa. Ochrana spočíva v HTTPS, bezpečných cookies, náhodných session ID, timeoutoch a ochrane pred XSS. Session riding, teda CSRF, zneužíva fakt, že prehliadač automaticky posiela cookies. Ochrana spočíva v CSRF tokenoch, SameSite cookies a kontrole pôvodu požiadavky.

URL tampering je manipulácia s parametrami v URL. Forceful browsing je priame zadávanie URL na skryté alebo chránené časti aplikácie. V oboch prípadoch platí, že server musí kontrolovať vstupy a oprávnenia. Nestačí skryť odkazy v používateľskom rozhraní.

HTML injection vzniká, keď sa používateľský vstup vloží do stránky ako HTML. Ak obsahuje JavaScript, ide o XSS. Ochrana spočíva v escapovaní výstupu, bezpečnom DOM API a nepoužívaní neovereného vstupu v `innerHTML` alebo `.html()`. SQL injection vzniká, keď sa používateľský vstup vloží do SQL dotazu ako kód. Ochrana spočíva v parametrizovaných dotazoch a prepared statements.

Pri validácii je bezpečnejší white-list prístup, ktorý povoľuje iba známe dobré hodnoty. Black-list prístup, ktorý zakazuje iba známe zlé hodnoty, je často neúplný a ľahko obíditeľný.